University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

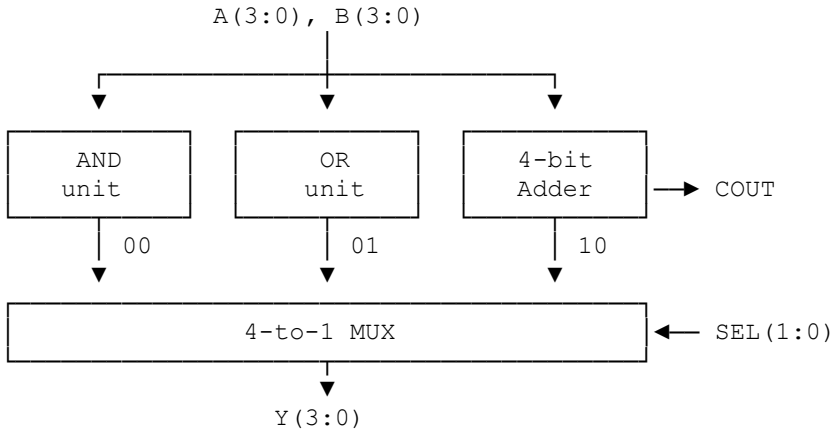
Mid-Term Lab Examination | Group B | Spring 2026

Duration	1 hr	**Total Marks**	20
Student Name	Md. Sahriar Asif	**ID**	22201111

Mid-Term Task: 4-bit ALU

Objective. Design and implement a 4-bit Arithmetic Logic Unit (ALU) in VHDL that supports exactly three operations: AND, OR,, and ADD. The ALU must be built from separate, reusable modules (logic units, a 4-bit adder, and a multiplexer) and combined in a top-level entity.

System Architecture



Functional requirement. The 2-bit select input SEL chooses which result is passed to the output Y. The AND and OR units operate bitwise on A and B, while the adder produces a 4-bit sum together with a carry-out. The ALU is purely combinational: Y and COUT must change whenever A, B, or SEL change. COUT must be 0 for every operation other than ADD.

Required Operation

SEL(1:0)	Operation	Y(3:0)	COUT
00	AND	Y ← A AND B	0
01	OR	Y ← A OR B	0
10	ADD	Y ← (A + B) mod 16	Carry-out of the 4-bit addition
11	Unused	Y ← 0000	0

Required Test Sequence

Your testbench must apply the following sequence. Allow each input combination to settle (for example, wait for 10 ns) before checking the outputs.

Step	SEL	A	B	Operation	Expected Y	COUT	Explanation
1	00	1100	1010	A AND B	1000	0	Bitwise AND
2	00	1111	0101	A AND B	0101	0	AND with all-ones passes B
3	01	1100	0011	A OR B	1111	0	Bitwise OR
4	01	1001	0100	A OR B	1101	0	Bitwise OR
5	10	0011	0101	3 + 5	1000	0	Simple addition
6	10	1111	0001	15 + 1	0000*	1	4-bit overflow
7	10	1010	0101	10 + 5	1111	0	Maximum 4-bit result
8	11	1010	0101	Unused	0000	0	Invalid select code

*** Note:** For Step 6, the mathematical result is 16, but a 4-bit output can represent only 0–15. Therefore, Y = 0000 and the discarded carry appears on COUT = 1.

Implementation Requirements

- Use VHDL and IEEE.STD_LOGIC_1164.ALL.
- Ports: A, B : in STD_LOGIC_VECTOR(3 downto 0); SEL : in STD_LOGIC_VECTOR(1 downto 0); Y : out STD_LOGIC_VECTOR(3 downto 0); COUT : out STD_LOGIC.
- Build the 4-bit adder structurally from 1-bit full adders (ripple-carry). Do not use the "+" operator or an integer/numeric_std based adder as a replacement.
- The design must be modular: separate entities for the AND unit, OR unit, full adder, 4-bit adder, and 4-to-1 multiplexer, connected by a structural top-level ALU.
- The ALU must be combinational; no clock or process-based registers are allowed.
- SEL = "11" must produce Y = "0000" and COUT = '0'.
- The submitted testbench must apply all eight steps of the required test sequence and use ASSERT statements to compare the actual output with the expected output.

Submission

While submitting to the classroom prepare the report from the third page of this template including the current template in the first and second page. From the third page write your report including but not limited to the following points.

- Attach your entity code and testbench code
- Attach your output visuals (waveform showing all eight test steps)
- Challenges faced during the experiment
- Conclusion

Design and Simulation of a 4-bit Arithmetic Logic Unit (ALU) in VHDL

1. Objectives

- To design a 4-bit ALU in VHDL that performs AND, OR and ADD, built from separate reusable modules (AND unit, OR unit, a ripple-carry 4-bit adder made of 1-bit full adders, and a 4-to-1 multiplexer) connected by a structural top-level entity.
- To verify the design with a testbench that applies all eight required test steps and checks Y and COUT using ASSERT statements, including the 4-bit overflow case and the unused select code (SEL = "11").
- To generate the RTL schematics of the 4-bit adder, the 4-to-1 MUX and the 4-bit ALU and to confirm the simulated waveform against the expected outputs.

2. Entity Code and Testbench Code

The VHDL entity code of the 4-bit adder, the 4-to-1 MUX and the 4-bit ALU, each with its testbench, is given below.

4-bit Adder (ripple-carry, structural)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity low_nibble_4bit is
    Port ( A    : in  STD_LOGIC_VECTOR (3 downto 0);
           B    : in  STD_LOGIC_VECTOR (3 downto 0);
           CIN  : in  STD_LOGIC;
           SUM  : out STD_LOGIC_VECTOR (3 downto 0);
           COUT : out STD_LOGIC);
end low_nibble_4bit;
architecture Structural of low_nibble_4bit is

    component full_adder
        Port ( A     : in  STD_LOGIC;
               B     : in  STD_LOGIC;
               CIN   : in  STD_LOGIC;
               SUM   : out STD_LOGIC;
               COUT  : out STD_LOGIC);
    end component;

    -- internal carry chain: carry(0) = CIN, carry(4) = COUT
    signal carry : STD_LOGIC_VECTOR (4 downto 0);

begin

    carry(0) <= CIN;
    COUT     <= carry(4);

    GEN_ADD: for i in 0 to 3 generate
        FA: full_adder port map (
            A    => A(i),
            B    => B(i),
            CIN  => carry(i),
            SUM  => SUM(i),
            COUT => carry(i+1)
        );
    end generate GEN_ADD;

end Structural;
```

4-bit Adder Testbench

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY low_nibble_4bit_tb IS
END low_nibble_4bit_tb;

ARCHITECTURE behavior OF low_nibble_4bit_tb IS

    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT adder_4bit
    PORT(
         A    : IN  std_logic_vector(3 downto 0);
         B    : IN  std_logic_vector(3 downto 0);
         CIN  : IN  std_logic;
         SUM  : OUT std_logic_vector(3 downto 0);
         COUT : OUT std_logic
        );
    END COMPONENT;

            -- Inputs
    signal A   : std_logic_vector(3 downto 0) := (others => '0');
    signal B   : std_logic_vector(3 downto 0) := (others => '0');
    signal CIN : std_logic := '0';

    -- Outputs
    signal SUM  : std_logic_vector(3 downto 0);
    signal COUT : std_logic;

BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: adder_4bit PORT MAP (
          A    => A,
          B    => B,
          CIN  => CIN,
          SUM  => SUM,
          COUT => COUT
        );

                   -- Stimulus process
    stim_proc: process
    begin

        -- Test 1: 0000 + 0000 + 0 = 0000
        A <= "0000";
        B <= "0000";
        CIN <= '0';
        wait for 10 ns;

        -- Test 2: 0001 + 0010 = 0011
        A <= "0001";
        B <= "0010";
        CIN <= '0';
        wait for 10 ns;

                       -- Test 3: 0101 + 0011 = 1000
        A <= "0101";
        B <= "0011";
        CIN <= '0';
        wait for 10 ns;
```

```
        -- Test 4: 1111 + 0001 = 0000, COUT = 1
        A <= "1111";
        B <= "0001";
        CIN <= '0';
        wait for 10 ns;

        -- Test 5: 1010 + 0101 + 1 = 0000, COUT = 1
        A <= "1010";
        B <= "0101";
        CIN <= '1';
        wait for 10 ns;

        wait;
    end process;

END;
```

4-to-1 Multiplexer

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux_4bit is
    Port (
        I0 : in  STD_LOGIC_VECTOR(3 downto 0);
        I1 : in  STD_LOGIC_VECTOR(3 downto 0);
        I2 : in  STD_LOGIC_VECTOR(3 downto 0);
        I3 : in  STD_LOGIC_VECTOR(3 downto 0);
        S  : in  STD_LOGIC_VECTOR(1 downto 0);
        Y  : out STD_LOGIC_VECTOR(3 downto 0)
    );
end mux_4bit;

architecture Structural of mux_4bit is

begin

    Y <= I0 when S = "00" else
         I1 when S = "01" else
         I2 when S = "10" else
         I3;

end Structural;
```

4-to-1 Multiplexer Testbench

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux_4bit_tb IS
END mux_4bit_tb;

ARCHITECTURE behavior OF mux_4bit_tb IS

    COMPONENT mux_4bit
    PORT(
         I0 : IN  std_logic_vector(3 downto 0);
         I1 : IN  std_logic_vector(3 downto 0);
         I2 : IN  std_logic_vector(3 downto 0);
         I3 : IN  std_logic_vector(3 downto 0);
         S  : IN  std_logic_vector(1 downto 0);
         Y  : OUT std_logic_vector(3 downto 0)
```

```
        );
    END COMPONENT;

    signal I0 : std_logic_vector(3 downto 0) := (others => '0');
    signal I1 : std_logic_vector(3 downto 0) := (others => '0');
    signal I2 : std_logic_vector(3 downto 0) := (others => '0');
    signal I3 : std_logic_vector(3 downto 0) := (others => '0');
    signal S  : std_logic_vector(1 downto 0) := (others => '0');

    signal Y : std_logic_vector(3 downto 0);

BEGIN

    uut: mux_4bit PORT MAP (
        I0 => I0,
        I1 => I1,
        I2 => I2,
        I3 => I3,
        S  => S,
        Y  => Y
    );

    stim_proc: process
    begin

        -- Set input values
        I0 <= "0001";
        I1 <= "0010";
        I2 <= "0100";
        I3 <= "1000";

        -- Select I0
        S <= "00";
        wait for 10 ns;

        assert Y = "0001"
        report "MUX test 1 failed"
        severity error;

        -- Select I1
        S <= "01";
        wait for 10 ns;

        assert Y = "0010"
        report "MUX test 2 failed"
        severity error;

        -- Select I2
        S <= "10";
        wait for 10 ns;

        assert Y = "0100"
        report "MUX test 3 failed"
        severity error;

        -- Select I3
        S <= "11";
        wait for 10 ns;

        assert Y = "1000"
        report "MUX test 4 failed"
        severity error;

        report "All MUX test cases passed successfully."
```

```
        severity note;

        wait;
    end process;

END behavior;
```

4-bit ALU (top-level, structural)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity alu_4bit is
    Port (
        A    : in  STD_LOGIC_VECTOR(3 downto 0);
        B    : in  STD_LOGIC_VECTOR(3 downto 0);
        SEL  : in  STD_LOGIC_VECTOR(1 downto 0);
        Y    : out STD_LOGIC_VECTOR(3 downto 0);
        COUT : out STD_LOGIC
    );
end alu_4bit;

architecture Structural of alu_4bit is

    component low_nibble_4bit
        Port (
            A    : in  STD_LOGIC_VECTOR(3 downto 0);
            B    : in  STD_LOGIC_VECTOR(3 downto 0);
            CIN  : in  STD_LOGIC;
            SUM  : out STD_LOGIC_VECTOR(3 downto 0);
            COUT : out STD_LOGIC
        );
    end component;

    component mux_4bit
        Port (
            I0 : in  STD_LOGIC_VECTOR(3 downto 0);
            I1 : in  STD_LOGIC_VECTOR(3 downto 0);
            I2 : in  STD_LOGIC_VECTOR(3 downto 0);
            I3 : in  STD_LOGIC_VECTOR(3 downto 0);
            S  : in  STD_LOGIC_VECTOR(1 downto 0);
            Y  : out STD_LOGIC_VECTOR(3 downto 0)
        );
    end component;

    signal AND_RESULT : STD_LOGIC_VECTOR(3 downto 0);
    signal OR_RESULT  : STD_LOGIC_VECTOR(3 downto 0);
    signal ADD_RESULT : STD_LOGIC_VECTOR(3 downto 0);
    signal ADD_COUT   : STD_LOGIC;

    signal RESULT_INT : STD_LOGIC_VECTOR(3 downto 0);

begin

    -- Logic operations
    AND_RESULT <= A and B;
    OR_RESULT  <= A or B;

    -- Addition
    ADDER: low_nibble_4bit
        port map (
            A    => A,
```

```
            B    => B,
            CIN  => '0',
            SUM  => ADD_RESULT,
            COUT => ADD_COUT
        );

    -- Select operation
    MUX: mux_4bit
        port map (
            I0 => AND_RESULT,
            I1 => OR_RESULT,
            I2 => ADD_RESULT,
            I3 => "0000",
            S  => SEL,
            Y  => RESULT_INT
        );

    -- Connect result
    Y <= RESULT_INT;

    -- Carry-out only for ADD
    COUT <= ADD_COUT when SEL = "10" else
            '0';

end Structural;
```

4-bit ALU Testbench (all eight test steps with ASSERT statements)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY alu_4bit_tb IS
END alu_4bit_tb;

ARCHITECTURE behavior OF alu_4bit_tb IS

    COMPONENT alu_4bit
    PORT(
         A    : IN  std_logic_vector(3 downto 0);
         B    : IN  std_logic_vector(3 downto 0);
         SEL  : IN  std_logic_vector(1 downto 0);
         Y    : OUT std_logic_vector(3 downto 0);
         COUT : OUT std_logic
        );
    END COMPONENT;

    signal A    : std_logic_vector(3 downto 0) := (others => '0');
    signal B    : std_logic_vector(3 downto 0) := (others => '0');
    signal SEL  : std_logic_vector(1 downto 0) := (others => '0');

    signal Y    : std_logic_vector(3 downto 0);
    signal COUT : std_logic;

BEGIN

    uut: alu_4bit PORT MAP (
        A    => A,
        B    => B,
        SEL  => SEL,
        Y    => Y,
        COUT => COUT
    );
```

```
    stim_proc: process
    begin

        -- Step 1: AND
        SEL <= "00";
        A <= "1100";
        B <= "1010";
        wait for 10 ns;

        assert Y = "1000" and COUT = '0'
        report "Step 1 failed"
        severity error;

        -- Step 2: AND
        SEL <= "00";
        A <= "1111";
        B <= "0101";
        wait for 10 ns;


        assert Y = "0101" and COUT = '0'
        report "Step 2 failed"
        severity error;


        -- Step 3: OR
        SEL <= "01";
        A <= "1100";
        B <= "0011";
        wait for 10 ns;

        assert Y = "1111" and COUT = '0'
        report "Step 3 failed"
        severity error;


        -- Step 4: OR
        SEL <= "01";
        A <= "1001";
        B <= "0100";
        wait for 10 ns;

        assert Y = "1101" and COUT = '0'
        report "Step 4 failed"
        severity error;

        -- Step 5: ADD
        SEL <= "10";
        A <= "0011";
        B <= "0101";
        wait for 10 ns;

        assert Y = "1000" and COUT = '0'
        report "Step 5 failed"
        severity error;

        -- Step 6: ADD with overflow
        SEL <= "10";
        A <= "1111";
        B <= "0001";
        wait for 10 ns;


        assert Y = "0000" and COUT = '1'
```

```
        report "Step 6 failed"
        severity error;


        -- Step 7: ADD
        SEL <= "10";
        A <= "1010";
        B <= "0101";
        wait for 10 ns;

        assert Y = "1111" and COUT = '0'
        report "Step 7 failed"
        severity error;


        -- Step 8: Unused
        SEL <= "11";
        A <= "1010";
        B <= "0101";
        wait for 10 ns;

        assert Y = "0000" and COUT = '0'
        report "Step 8 failed"
        severity error;


        report "All test cases passed successfully."
        severity note;

        wait;

    end process;

END behavior;
```

3. Output Visuals

RTL Schematics

The RTL schematics generated for the 4-bit adder, the 4-to-1 MUX and the 4-bit ALU are shown below.

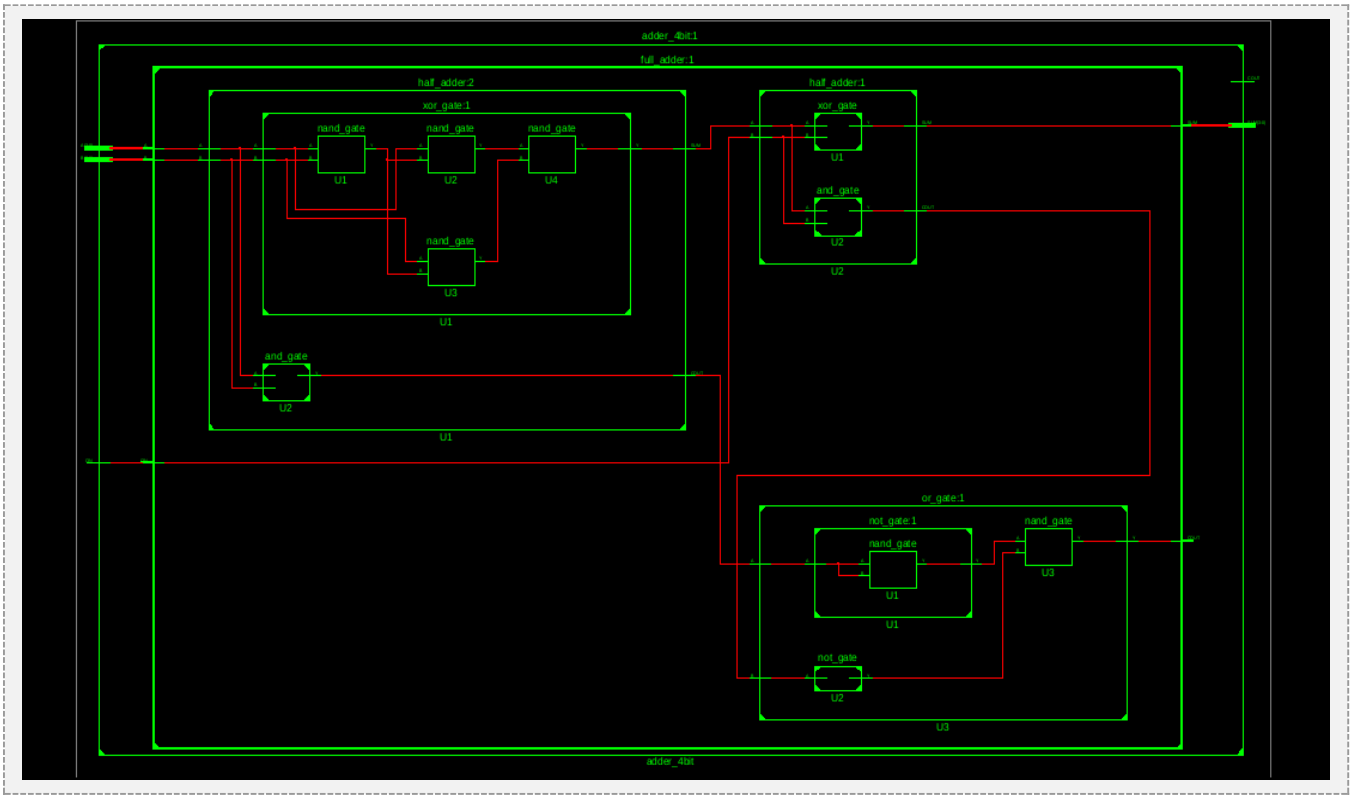


Figure 1: RTL schematic of the 4-bit Adder

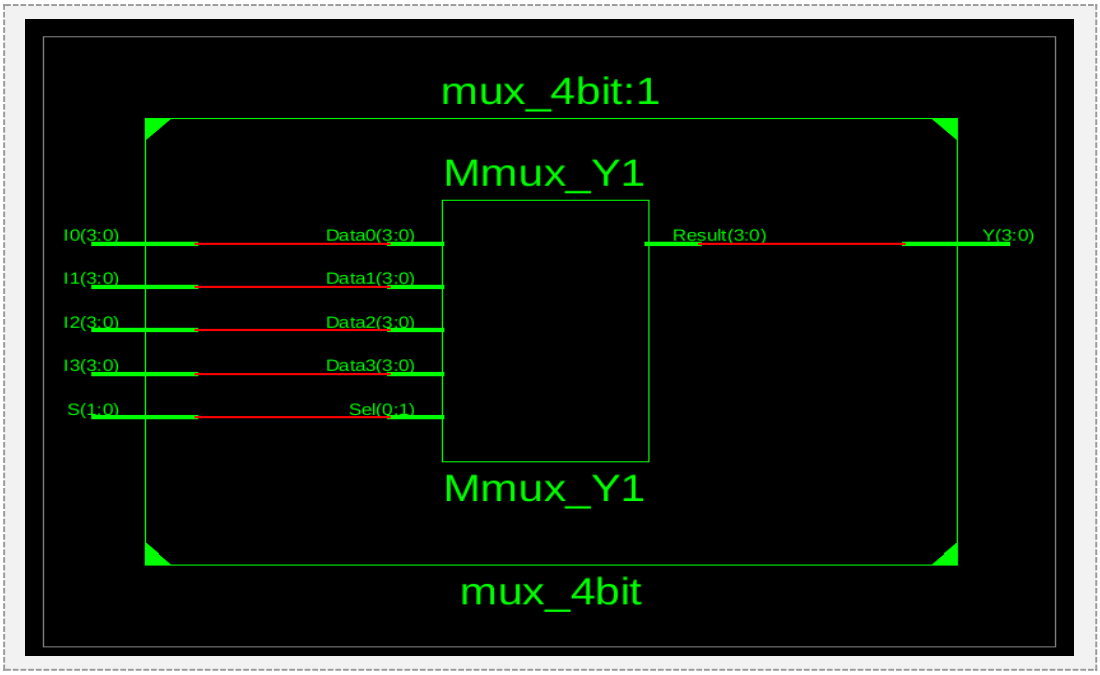


Figure 2: RTL schematic of the 4-to-1 MUX

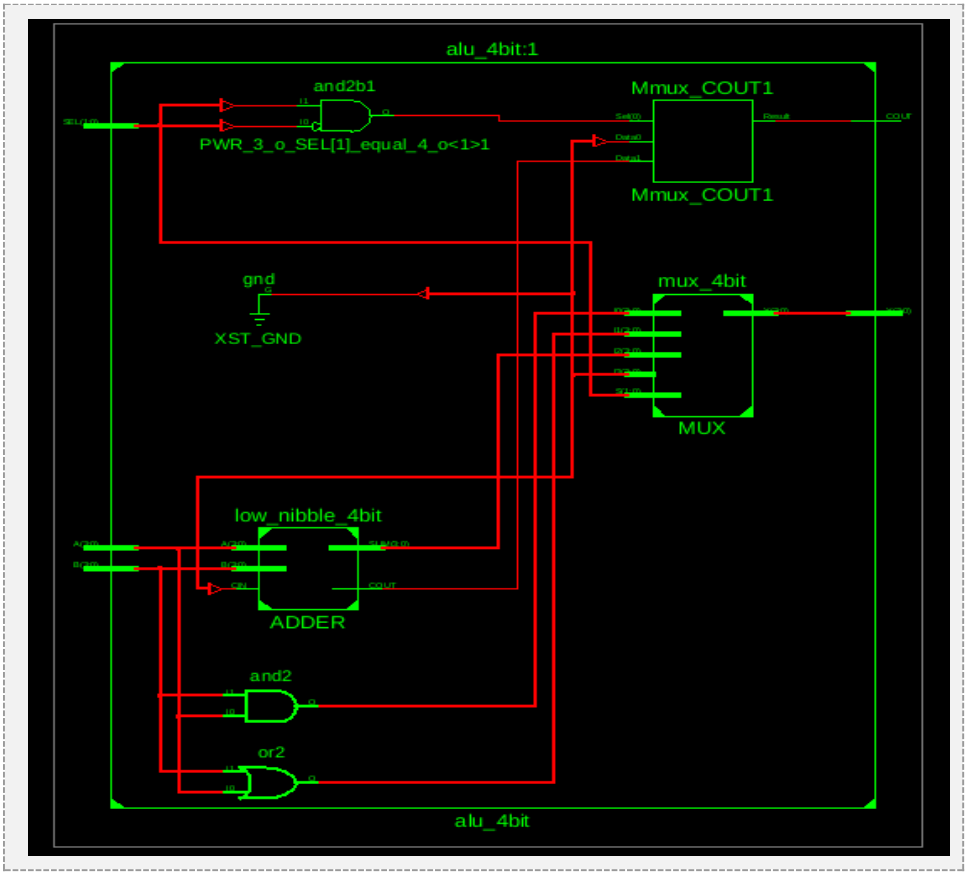


Figure 3: RTL schematic of the 4-bit ALU

Simulation Waveform

Waveform of 4-bit Adder, 4-bit Mux, 4-bit ALU are shown below respectively. The waveform below shows the output of the 4-bit ALU for all eight steps of the required test sequence. The inputs and expected outputs of each step are listed in the table that follows.

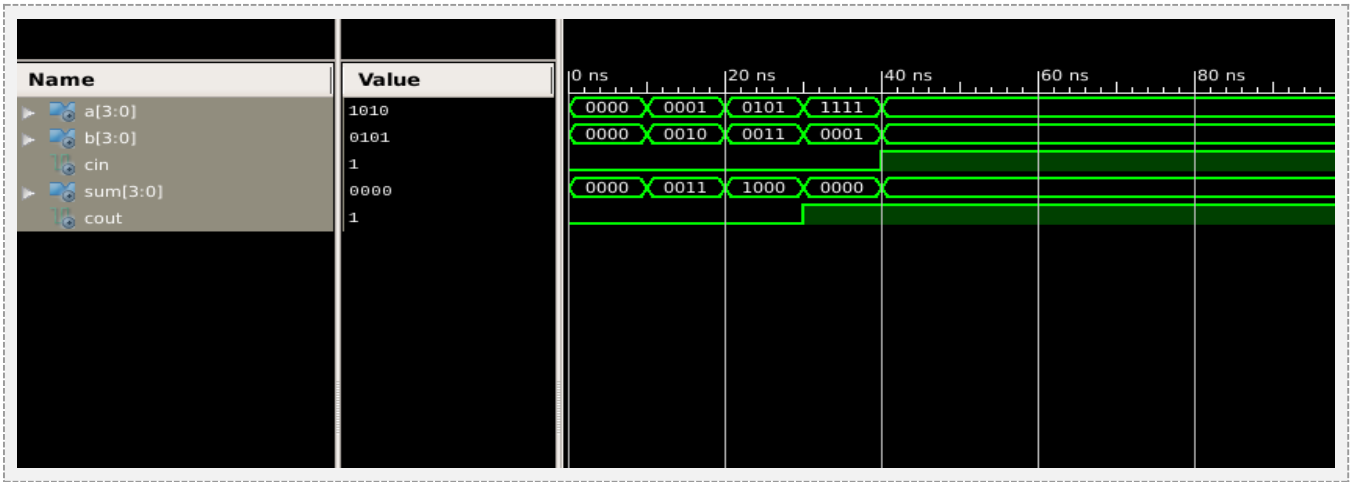


Figure 1: Simulation waveform of the 4-bit Adder

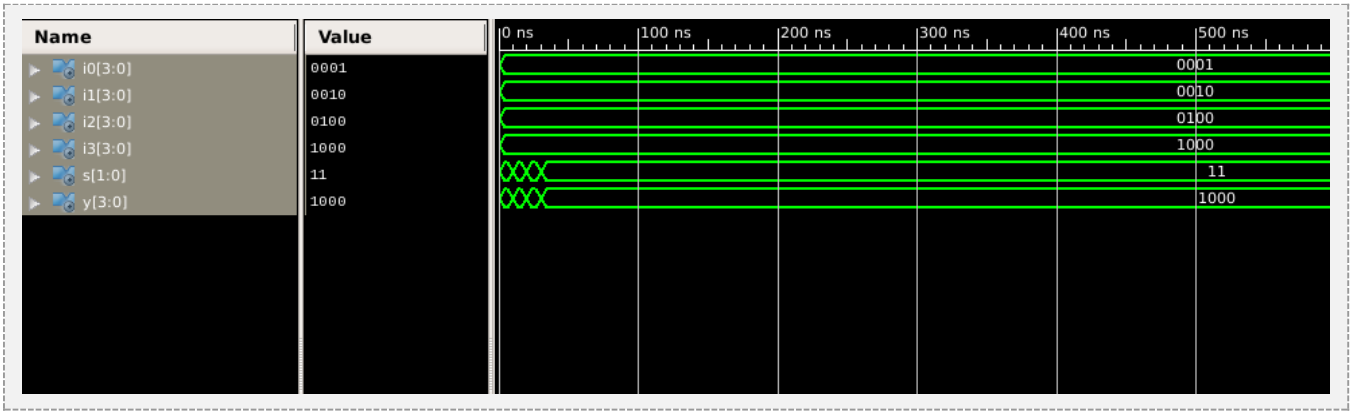


Figure 2: Simulation waveform of the 4-bit mux

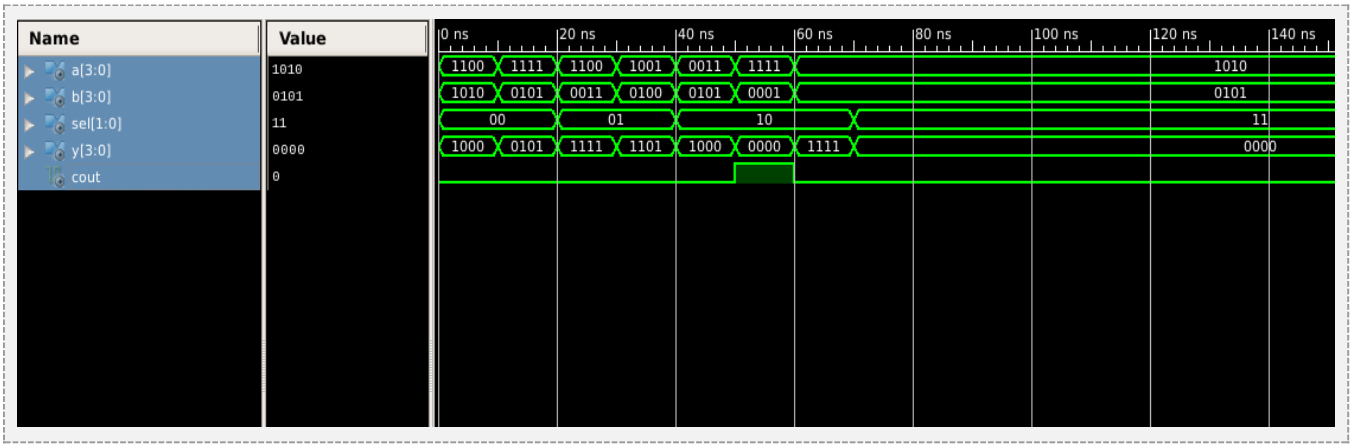


Figure 3: Simulation waveform of the 4-bit ALU (test steps 1 to 8)

Test sequence applied by the 4 bit ALU testbench (expected outputs)

Step	SEL	A	B	Operation	Expected Y	COUT
1	00	1100	1010	A AND B	1000	0
2	00	1111	0101	A AND B	0101	0
3	01	1100	0011	A OR B	1111	0
4	01	1001	0100	A OR B	1101	0
5	10	0011	0101	3 + 5	1000	0
6	10	1111	0001	15 + 1	0000	1
7	10	1010	0101	10 + 5	1111	0
8	11	1010	0101	Unused	0000	0

4. Challenges Faced During the Experiment

- **Ripple-carry connection.** Connecting the four 1-bit full adders structurally required internal carry signals between the stages. The carry-in of the first stage was set to '0', and the carry-out of the last stage was connected to COUT.
- **RTL schematic generation.** Initially, the RTL Schematic could not be opened because the required NGR file was not generated. The issue was resolved by restarting the Xilinx ISE.
- **COUT for different operations.** The 4-bit adder produces a carry-out, but COUT is required only for the ADD operation. Therefore, COUT was set to '0' for AND, OR, and the unused select code, and the adder carry-out was passed only when SEL = "10".
- **Unused select code.** The SEL = "11" condition was unused, so it was defined to produce Y = "0000" and COUT = '0'. This ensured that every select condition had a defined output.
- **Testbench and port mapping.** Correctly matching the port names and 4-bit vector widths between the modules was important to avoid compilation errors. The testbench also required a 10 ns wait between each test case and ASSERT statements to verify the expected Y and COUT values.

5. Conclusion

A 4-bit ALU supporting AND, OR, and ADD operations was designed and implemented in VHDL using a structural approach. The ALU used the previously developed ripple-carry 4-bit adder and a newly implemented 4-to-1 multiplexer, while the AND and OR operations were implemented directly in the top-level ALU. The testbench applied all eight required test cases, and the simulated Y and COUT values matched the expected results. The overflow case, 15 + 1, produced Y = 0000 with COUT = 1, while the unused SEL = "11" produced Y = 0000 and COUT = 0. The experiment demonstrated how previously tested modules can be combined to build a larger combinational circuit and how ASSERT statements can be used to verify the design automatically.